

Python Programming

TABLE OF CONTENT

- ❖ Module 1
 - Unit 1.1: Python Basics
 - Unit 1.2: Python Control Structures
 - Unit 1.3: Python Functions and Modules
 - Unit 1.4: Data Structures in Python
- ❖ Module 2
 - Unit 2.1: Object Oriented Programming Fundamentals
 - Unit 2.2:
 - Unit 2.3:
- ❖ Module 3
 - Unit 3.1:
 - Unit 3.2:
- ❖ Module 4
 - Unit 4.1:
 - Unit 4.2:
 - Unit 4.3:

➤ Unit 1.1: Python Basics

Python and it's Syntax

- **Overview**

- Python is a powerful, user-friendly programming language. It stands out for its readability and extensive libraries.
- Its versatility extends to many applications such as web development, data science, artificial intelligence, and automation.
- Python is an interpreted language which means its source code is executed line by line by an interpreter, as opposed to being compiled into machine code before execution.

- **History**

- Python was initially designed by Guido Van Rossum in 1991 and developed by Python Software Foundation. It was mainly developed for emphasis on code readability, and its syntax allows programmers to express concepts in fewer lines of code.
- Python was a successor to the ABC programming language which was capable of exception handling and interfacing with the Amoeba operating system.
- On October 16 of 2000, Python 2.0 was released and it had many major new features including cycle-detecting garbage collector for memory management and support for Unicode.
- Python 3.0 also called Py3k was released on 3 December 2008. It was designed to rectify the fundamental flaw of the language.

- **What is python?**

High Level Language	Interpreted Language	Object Oriented Language
• Python uses common English words in its code, making it readable. • Python is known for its simplicity, readability, and ease of use, making it a popular choice for beginners as well as experienced developers.	• Python is processed at runtime by the interpreter without needing a compiler. • Unlike compilers that translate the entire code before execution, Python's interpreter processes code line by line at runtime.	• In Python, everything is treated as an object, mirroring real-world entities. • Objects possess both characteristics and actions.

- **Programming Syntax:**

- **Python Print Function:** The **print()** function prints the specified message to the screen, or to any other standard output device.
e.g. - `print('Hello World!')`
`print("Hello World!")`
- **Multiple Statements in One Line:** For writing multiple statements in one line, we can separate them with a semicolon.
e.g.- `print('hi'); print("Data Analyst")`
Note: When you write one or more such Python statements then it is called as **Python Script**.
- **Multi-line statement:** When Python encounters a 'backslash' at the end of a line, it treats it as a line continuation, allowing the string to span multiple lines.
e.g.- `print("hello\`
`World")`

- **Python Comments:**

Comments in a Python program serve as annotations that provide clarity on the purpose and functionality of specific code segments. This practice significantly aids in comprehending the program's logic and serves as a valuable documentation tool for both the original coder and potential collaborators.

Single-line comments in Python start with '#' and extend to the line's end. For *multiline comments*, triple-quotes (""") or single triple quotes (''') are used when longer descriptions are needed.

e.g.- #This is a single line comment.
 """This is multi line comment.(using double quotes)"""
 '''This is multi line comment.(using single quotes)'''

Python Essentials

- **Python Keywords:**

- Keywords are the reserved words in Python.
- They are used to define the syntax and structure of the Python language.
- In Python, keywords are case sensitive.
- All the keywords except True, False and None are in lowercase and they must be written as they are.
- We cannot use a keyword as a variable name, or any other identifier

and	del	for	def	False	finally	in	is	pass	lambda
print		with	c	raise	import	not	or	while	try
assert		Break		lass	yield	continue	exec	except	if
else		global		as	elif	from	None	nonlocal	
return									

- **Python Identifiers:**

A Python identifier is a name used to identify a variable which stores some value, or any object.

There are some rules to follow while choosing an identifier:

- An identifier may only begin with A-Z, a-z, or an underscore(_)
- Python is case-sensitive. "var1" and "Var1" are two different identifiers.
- A reserved keyword may not be used as an identifier.
- We cannot use special symbols like !, @, #, \$, % symbols in our identifier.

Valid	Invalid
A	A@
Al1	1name
_number	Var 1
Num_1	global
Student_id	(global is keyword)
productName	

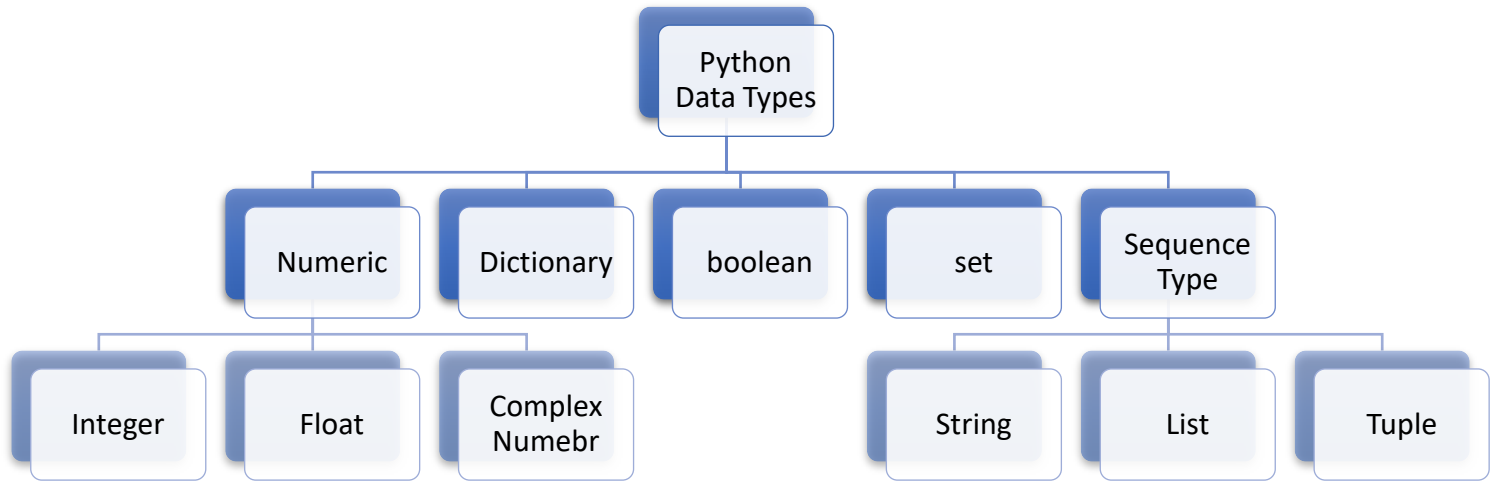
- **Variables:**

- Reserved memory locations to store values are called variables.
- Unlike other programming languages, Python has no command for declaring a variable.
- Variables are assigned with values.
- Python is not "statically typed". We do not need to declare variables before using them, or declare their type. A variable is created the moment we first assign a value to it

- When multiple variables point to the same memory location in Python, it means that these variables are referencing or pointing to the same underlying data in the computer's memory.
e.g.- num1=100, num2=100, num3=num2
Here variables num1, num2, and num3 are all pointing to the *same memory location*.

- **Data Types:**

Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data.



The table shows classification of python data types with examples:

Data Type	Description	Example
Numeric	Data types for storing numbers	
	• int: Whole numbers	0, 99
	• float: Floating-point numbers with decimal places	3.14, 9.18
	• complex: Complex numbers with real and imaginary parts	1+2i, 8-3i
Dictionary	Store key-value pairs	{'apple': 'red', 'banana': 'yellow'}
Boolean	True or False values	True, False
Set	Collections of unique items	
	• String: Sequences of characters for text data	"Hello, world!", 'This is a string.'
	• List: Mutable collections of items	['apple', 'banana', 'orange']
	• Tuple: Immutable collections of items	('apple', 'banana', 'orange')

- **Numeric:**

Numeric value can be integer, floating number or even complex numbers. These values are defined as int, float and complex class in Python.

- **Integers** - This value is represented by **int class**. It contains positive or negative whole numbers (without fraction or decimal). For example-66
- **Float** – This value is represented by **float class**. It is a real number with floating point representation. It is specified by a decimal point. Optionally, the character e or E followed by a positive or negative integer may be appended to specify scientific notation. For example-60.9
- **Complex Numbers** –Complex number is represented by **complex class**. It is specified as **(real part) + (imaginary part)i**. For example – 2+3i
e.g.(code) –

```
# demonstrating numeric value
a = 5
print("type of a: ", type(a))
b = 2
print("\ntype of b: ", type(b))
c=1+6i
print("\ntype of c: ", type(c))
```

Output:

```
type of a:  <class 'int'>
type of b:  <class 'float'>
type of c:  <class 'complex'>
```

Note – **type()** function is used to determine the type of data type.

- **Sequence Data Type:**

In Python, sequence is the ordered collection of similar or different data types. Sequences allows to store multiple values in an organized and efficient fashion. There are several sequence types in Python - String, List, Tuple

- **String:**

Strings are arrays of bytes representing Unicode characters. A string is a collection of one or more characters put in a single quote, double-quote or triple quote. In python there is no character data type, a character is a string of length one. It is represented by **str class**.

e.g.(code) –

```
String1 = "Program 1"
print("first String: ",String1)
```

```
String2 = "Program 2"
print("second String: ",String2)
```

```
String3 = "Anything"
print("third String: ",String3)
```

Output:

```
first String:  Program 1
second String:  Program 2
third String:  Anything
```

- **Lists:**

Lists are just like the arrays, declared in other languages which is a **ordered collection** of data. It is very flexible as the items in a list do not need to be of the same type.

- **Tuple:**

Just like list, tuple is also an **ordered collection** of Python objects. The only difference between type and list is that tuples are immutable i.e. tuples **cannot be modified** after it is created. It is represented by tuple class.

- **Boolean:**

The Boolean data type is either True or False. In Python, Boolean variables are defined by the *True and False keywords*. It is denoted by the **class bool**.

Note – True and False with capital 'T' and 'F' are valid booleans otherwise Python will throw an error. e.g.(code) –

```
# demonstrate Boolean type
print(type(True))
print(type(False))
print(type(true))
```

Output:

```
<class 'bool'>
<class 'bool'>
NameError
Traceback (most recent call last)
<ipython-input-1-6fd314972159> in <module>----> 1
print(type(true))
NameError: name 'true' is not define
```

- **Escape Sequences:**

It is a combination of characters that represents a special character or instruction. Escape sequences begin with a backslash \. They are used to include characters in strings that would otherwise be difficult to represent directly, such as newline characters, tabs, or quotation marks.

Example:

- \n: Newline character (moves the cursor to the next line).
- \t: Tab character (inserts a horizontal tab).
- \' : Single quote character.
- \" : Double quote character.
- \\ : Backslash character

Python Operators and Expressions

- **Python Operators:**

Operators are symbols or characters that are used to perform various operations on one or more values or variables. Following are their main types:

1. Arithmetic Operators:

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, etc.

Operators	Examples	Description
+ (Addition)	a, b=9, 2 sum = a+b	Add two operands. Two variables added together.
- (Subtraction)	a, b=9, 2 sub= a-b	Variable b is subtracted from variable a.
/ (Division)	a, b=9, 2 c= a/b	The quotient of two variables a & b on dividing is assigned to c.
* (Multiplication)	a, b=9, 2 c= a*b	Multiplies two operands together.
% (Modulo)	a, b=9, 2 c= a%b	c now stores the remainder on dividing a by b.
** (Exponentiation)	a=2 print(a**3) Output: 8	Raises a number to a power. 2 raised to the power 3 will get printed.

2. Assignment Operators:

These operators are used to assign values to variables. The following table shows the list of the assignment operators:

Operators	Examples	Description
=	a=10	Operator that assigns the value on the right side to the variable on the left side.
+=	a, b= 10, 20 a+=b print(a) Output: 30	Shorthand for adding the left-hand operand with the right one. It is equal to a=a+b.
-=	a, b = 20, 10 a-= b	It subtracts the right-hand operand from the left-hand operand. It is equal to a=a-b.
/=	a=20 a/=2	Variable a is stored with the value of variable a divided by 2.
=	a, b = 10, 20 a= b	Variable a is stored with the value of variable multiplied by b.
%=	a=20 a%=3	a%=3 computes the remainder of a divided by 3 and stores the result back into a.

Operators	Examples	Equivalent to
<code>/=</code>	<code>a /= 5</code> (Output: int/float)	<code>a = a/5</code>
<code>//=</code>	<code>a //= 5</code> (Output: int)	<code>a = a//5</code> (floor division)
<code>**=</code>	<code>a **= 5</code>	<code>a = a**5</code>
<code>&=</code>	<code>a &= 5</code>	<code>a = a&5</code>
<code> =</code>	<code>a = 5</code>	<code>a = a 5</code>
<code>^=</code>	<code>a ^= 5</code>	<code>a = a^5</code>

3. Comparison Operators:

Comparison operators are used to compare variables and values. Some of the comparison operators are:

Operators	Examples	Description
<code>==</code>	<code>a=3, b=4</code> <code>print(a==b)</code> Output: False	It returns True only if the values of a and b are equal, otherwise returns False.
<code>!=</code>	<code>a=3, b=4</code> <code>print(a!=b)</code> Output: True	It returns True if values of a and b are not equal, otherwise returns False.
<code>></code>	<code>a=3, b=4</code> <code>print(a>b)</code> Output: False	It returns True if the value of a is larger than the value of b, otherwise returns False.
<code><</code>	<code>a=3, b=4</code> <code>print(a<b)</code> Output: True	It returns True if the value of a is smaller than the value of b, otherwise returns False.
<code>>=</code>	<code>a=4, b=4</code> <code>print(a>=b)</code> Output: True	It returns True if the value of a is greater than or equal to the value of b, otherwise returns False.
<code><=</code>	<code>a=3, b=4</code> <code>print(a<=b)</code> Output: True	It returns True if the value of a is less than or equal to the value of b, otherwise returns False.

4. Logical Operators:

Logical operators are used to perform logical operations on one or more boolean values (i.e., True or False). They are used to combine or manipulate boolean expressions to produce a single boolean result.

There are three different types of logical operators:

Operators	Examples	Description
and	<code>x = 5</code> <code>print(x > 3 and x < 10)</code> Output: True	If both operands are True, this operator returns True; otherwise, it returns False.
or	<code>x = 5</code> <code>print(x > 3 or x < 4)</code> Output: True	This operator returns True if either of the operands is true and False otherwise.
not	<code>x = 5</code> <code>print(not(x > 3 and x < 10))</code> Output: False	The value of a boolean operand is inverted by this operator. It returns False if the operand is True, and True if the operand is False.

5. Bitwise Operators:

Bitwise operators act on operands as if they were strings of binary digits. They operate bit by bit.

Consider the table given below for two operands `a=40` and `b=20`

Operators	Examples	Description
& (Binary AND)	(a & b)=4 (means 100)	Operator copies a bit to the result if it exists in both operands.
 (Binary OR)	(a b) = 6 (means 110)	It copies a bit if it exists in either operand.
^ (Binary XOR)	(a ^ b) = 2 (means 10)	It copies the bit if it is set in one operand but not both.
~ (Binary Ones Complement)	(~a) = -5 (means -101)	It is unary and has the effect of 'flipping' bits.
<< (Binary Left Shift)	a << 1 = 8 (means 1000)	The left operands value is moved left by the number of bits specified by the right operand.
>> (Binary Right Shift)	a >> 1 = 2 (means 10)	The left operands value is moved right by the number of bits specified by the right operand.

6. Special Operators:

▪ Identity Operators:

is and **is not** are the identity operators in Python. They are used to check if two values (or variables) are located on the same part of the memory. Two variables that are equal does not imply that they are identical.

Operator	Example	Meaning
is	x is True	True if the operands are identical (refer to the same object)
is not	x is not True	True if the operands are not identical (do not refer to the same object)

▪ Membership Operators:

in and **not in** are the membership operators in Python. They are used to test whether a value or variable is found in a sequence (*string, list, tuple, set and dictionary*).

Operator	Example	Meaning
in	5 in x	True if value/variable is found in the sequence
not in	5 not in x	True if value/variable is not found in the sequence

○ Operator Precedence:

Python operator precedence is a set of rules that determine the order in which operators are evaluated in an expression. This is important because it can affect the result of an expression. Python's operator precedence follows the **PEMDAS** rule for arithmetic expressions.

P - stands for parentheses.

E - stands for exponentiation.

M - stands for multiplication.

D - stands for division.

A - stands for addition.

S - stands for subtraction.

operator precedence from lowest at bottom to highest at the top:

S.No.	Operator	Description
1	**	Exponentiation (raise the power) (HIGHEST)
2	~ + -	Complement, unary plus and minus (method names for the last two are +@ and -@)
3	*/%//	Multiply, divide, modulo and floor division
4	+ -	Addition and subtraction
5	>> <<	Right and left bitwise shift
6	&	Bitwise 'AND'
7	^	Bitwise exclusive 'OR' and regular 'OR'
8	<= < > >=	Comparison operators
9	<>== !=	Equality operators

10	=% /= //= -= += *= **==	Assignment operators
11	is; is not	Identity operators
12	in ; not in	Membership operators
13	Not or and	Logical operators (LOWEST)

○ **input() function:**

The **input()** function is used to get user input from the keyboard. It displays an optional message (prompt) to the user, waits for them to type something, and stores their input as a string.

Look at this example:

```
User_name = input('Please enter your name:')
user_age = input('Please enter your age:')
print(' Hello, '+user_name+'!')
print('You are '+user_age+' years old.')
```

Output:

```
Please enter your name:
Harsh
Please enter your age:    12
Hello, Harsh!
You are 12 years old.
```

Python Strings

1. String Operators:

Operator	Description	Example (a= 'Data' and b = 'Analysts')
+	Concatenation - Adds values on either side of the operator	a + b will give DataAnalysts
*	Repetition - Creates new strings, concatenating multiple copies of the same string	b*2 will give - AnalystsAnalysts
[]	Slice - Gives the character from the given index	a[1] will give – a
[:]	Range Slice - Gives the characters from the given range	b[0:3] will give Ana
in	Membership - Returns true if a character exists in the given string	'D' in a will give True
not in	Membership -Returns true if a character does not exist in the given string	'l' not in b will give False
r/R	Raw String - Suppresses actual meaning of Escape characters. The syntax for raw strings is exactly the same as for normal strings with the exception of the raw string operator	c = "1" + "\t" + "hello" d = "2" + r"\t" + "all" print(c); print(d) Output- 1 hello 2\tall

2. String Formatting:

Python uses C-style string formatting to create new, formatted strings using string format operator.

Character	Description
%c	Character
%s	String conversion via str() prior to formatting.
%i	Signed decimal integer.
%d	Signed decimal integer.
%u	Unsigned decimal integer.
%o	Octal integer.
%x & %X	Hexadecimal integer using lowercase & uppercase letters.
%e & %E	Exponential notation with lowercase & uppercase letters.
%f	Floating point real number.
%g	The shorter of %f and %e.
%G	The shorter of %f and %E.

e.g.(code):-

```
name = 'Data Analysts'
print("Hello %s" % name)
x=1; var='Data Analysts'
print('I am printing %s then %d' % (var,x))
```

Output:

```
Hello Data Analysts
I am printing Data Analysts then 1
```

3. **String Format Method:**

The format method allows you to format. At the places, you want to put values, put 0,1,2,.. in curly braces. Call the format method on the string and mention the identifiers in the parameters.

e.g.(code):-

```
x=1; var='Data Analysts'
print('I am printing {0} then {1}'.format(var,x))
```

Output:

```
I am printing Data Analysts then 1
```

○ **F – String:**

f-strings are string literals that have an f at the beginning and curly braces containing expressions that will be replaced with their values. The expressions are evaluated at runtime. f-strings are faster than both % formatting and str.format().

As f-strings are evaluated at runtime, you can put any and all valid Python expressions in them.

e.g.(code):-

```
x=1; var='Python for Data Science'
print(f' I am printing {var} then {x}')
f'{20*2}
```

Output:

```
I am printing Data Analysts then 1
40
```

Note- We can also use capital F instead of.

○ **Multiline F- Strings:**

The message variable utilizes a multi-line f-string enclosed within parentheses. Each line of the f-string represents a segment of the final message, incorporating expressions within curly braces to dynamically insert the values of the corresponding variables.

e.g.(code):-

```
name = 'Sachin Tendulkar'
profession = 'cricketer'
country = 'India'
message = (
f'Hello {name}. '
f'You are a {profession}. '
f'You are from {country}. '
)
print(message)
```

Output:

```
'Hello Sachin Tedulkar. You are a
cricketer. You are from India.'
```

➤ Unit 1.2: Python Control Structures

1.2.1 Python Conditionals

- **Conditional Statements:**

Conditional statements are used to make decisions in Python based on certain conditions. These are essential elements of control structures.

- **if statement:**

The if statement is used to execute a block of code if a specified condition is True.

e.g.(code):-

```
x = 10
if x > 5:
    print('x is greater than 5')
```

- **else statement:**

The else statement is used in conjunction with if. It specifies a block of code to be executed if the condition in the if statement is False.

e.g.(code):-

```
x = 3
if x > 5:
    print('x is greater than 5')
else:
    print('x is not greater than 5')
```

- **elif statement:**

The elif statement is used for handling multiple conditions. It allows you to check alternative conditions when the initial if condition is False.

e.g.(code):-

```
x = 5
if x > 5:
    print('x is greater than 5')
elif x == 5:
    print('x is equal to 5')
else:
    print('x is less than 5')
```

- **Ternary (conditional) operator:**

The ternary operator provides a concise way of writing simple if-else statements. It's useful for assigning values to variables based on conditions.

e.g.(code):-

```
x = 7
result = 'x is greater than 5' if
x > 5 else 'x is not greater than 5'
print(result)
```

1.2.2 Loops and Iteration

- **Loops and Iteration:**

Loops are used to execute a block of code repeatedly. They are essential for iterating over data structures or for performing repetitive tasks. Let us discuss types of Loops and iterations:

- **for loop:**

The for loop is used to iterate over a sequence (e.g., lists, strings) and execute a block of code for each item in the sequence.

e.g.(code):-

```
string = 'Data'
for char in string:
    print(char)
```

- **while loop:**

The while loop continues execution while a specified condition remains True.

e.g.(code):-

```
count = 1
while count <= 3:
    print(count)
    count += 1
```

- **Loop control statements (break and continue):**

break is used to exit a loop prematurely.

continue is used to skip the rest of the current iteration and proceed to the next one.

e.g.(code):-

Using 'break' to exit the loop prematurely

```
count = 1
while count <= 5:
    print("Current count:", count)
    if count == 3:
        break # Exit the loop
```

when count is 3

```
count += 1
```

Using 'continue' to skip an iteration

```
count = 1
while count <= 5:
    if count != 3:
        print("Current count:", count)
    count += 1
```

Output:

```
Current count: 1
Current count: 2
Current count: 3
Current count: 1
Current count: 2
Current count: 4
Current count: 5
```

- **Nested loops:**

Nested loops involve using loops within loops for complex iterations.

e.g.(code):-

```
i = 1
while i <= 2:
    j = 1
    while j <= 2:
        print(i, "x", j, "=", i * j)
        j += 1
    i += 1
    print('\n')
```

Output:

```
1 x 1 = 1
1 x 2 = 2

2 x 1 = 2
2 x 2 = 4
```

- **Range Function:**

The **range()** function generates a sequence of integers, commencing at 0 by default and incrementing by 1, concluding before a specified value.

e.g.(code):-

```
#1 for i in range(5):
    print(i)

#2 for i in range(5,8):
    print(i)
```

Output:

```
#1          #2
0 1 2 3 4   5 6 7
```

➤ Unit 1.3: Python Functions and Modules

1.3.1 Python Functions

- **Types of Functions:**

- **Built-in Functions:**

Built-in functions are functions that come pre-defined in Python. These functions are part of the Python standard library, and you can use them without having to define them yourself.

Some common examples of built-in functions are:

- `print()` Used to display output on the screen.
- `len()` Return the length of a sequence (e.g., a string, list, or tuple).
- `type()` Return the type of an object.
- `range()` Generates a sequence of numbers.
- `input()` Reads input from the user.

- **User Defined Functions:**

User-defined functions are functions that you create in your Python Code. These functions are defined using the `def` keyword, followed by a function name and a block of code that specifies what the function does. User-defined functions are useful for encapsulating a specific set of operations into a reusable unit. Here's the basic syntax for defining it:

```
def function_name(parameters):
```

```
    #Function code
```

```
    return result
```

e.g.(code):-

```
#Without argument/parameters.
```

```
def hello():
```

```
    print("Hello, World!")
```

```
hello()
```

```
#With arguments/parameters.
```

```
def func(name, job):
```

```
    print(name,"is a", job)
```

```
func('John','worker')
```

Output :

#1

Hello, World!

#2

John is a worker

- **Types of Arguments:**

Python handles function arguments in a very flexible manner, compared to other languages. It supports multiple types of arguments in the function definition.

- **Positional Arguments:**

Positional arguments values are copied to their corresponding parameters in order.

The only downside of positional arguments is that you need to pass arguments in the order in which they are defined.

e.g.(code):-

```
def func(name, job):
```

```
    print(name,"is a", job)
```

```
func('John','worker')
```

- **Keyword Arguments:**

To avoid positional argument confusion, you can pass arguments using the names of their corresponding parameters. The order of the arguments no longer matters because arguments are matched by name, not by position.

e.g.(code):-

```
# Keyword arguments can be put in any order
```

```
def func(name, job):
    print(name, 'is a', job)
func(name='John', job='data analyst')
func(job='data analyst', name='jhon')
```

Output:

```
Jhon is a data analyst
Jhon is a data analyst
```

○ **Default Arguments:**

You can specify default values for arguments when defining a function. The default value is used if the function is called without a corresponding argument Defaults allow you to make selected arguments optional.

e.g.(code):-

```
# Set default value ' data analyst ' to a 'job' parameter
def func(name, job=' data analyst '):
    print(name, 'is a', job)
func('John', 'manager')
func('john')
```

Output:

```
Jhon is a manager
Jhon is a data analyst
```

○ **Variable Length Arguments (*args and **kwargs):**

When you prefix a parameter with an asterisk *, it collects all the unmatched positional arguments into a tuple.

```
def print _ arguments(*args):
    print(args)
```

Output:

```
(1, 54, 60, 8, 98, 12)
```

The ** syntax is similar, but it only works for keyword arguments. It collects them into a new dictionary, where the argument names are the keys, and their values are the corresponding dictionary values.

```
def print _ arguments(**kwargs):
    print(kwargs)
```

Output:

```
{'name': 'Bob', 'age': 25, 'job': 'dev' }
```

○ **Return Value:**

To return a value from a function, simply use a return statement. Once a return statement is executed, nothing else in the function body is executed.

e.g.(code):-

```
# Return sum of two values
def sum(a, b):
    return a + b, a - b
x = sum(3, 4)
print(x)
```

Output:

```
(7, -1)
```

○ **Nested Functions:**

A Nested function is a function defined within other function. They are useful when performing complex task multiple times within another function, to avoid loops or code.

e.g.(code):-

```
def outer(a, b):
    def inner(c, d):
        return c + d
    return inner(a, b)
result = outer(2, 4)
print(result)
```

Output:

```
6
```

○ **Recursion:**

A recursive function is a function that calls itself and repeats its behavior until some condition is met to return a result.

e.g.(code):-

```
def countdown(num):  
    if num <= 0:  
        print('Stop')  
    else:  
        print(num)  
        countdown(num-1)  
countdown(5)
```

Output :

```
5  
4  
3  
2  
1  
stop
```

- **Scope of variables:**

In Python the scope of a variable refers to the region of the program where the variable is accessible or can be modified. The scope determines the visibility and lifetime of a variable. Python encompasses various variable scopes, including local, enclosing, global, and built-in

- **Local Scope:**

- Variables defined within a function are considered local to that function.
- They are only accessible within the function and not outside of it.
- Once the function execution completes, local variables are usually destroyed.

```
def example_function():  
    x = 10 # local variable  
    print(x)  
example_function()
```

- **Enclosing (or Non-local) Scope:**

- This scope is relevant for nested functions.
- If a variable is not found in the local scope, Python looks for it in the enclosing (containing) function's scope.
- The variable is not global but is in a scope enclosing the local scope.

```
def outer_function():  
    y = 20 # enclosing scope variable  
    def inner_function():  
        print(y) # y is accessible in the enclosing scope  
    inner_function()  
outer_function()
```

- **Global Scope:**

- Variables defined at the top level of a script or module are considered global.
- They are accessible throughout the entire module.
- Global variables can be accessed and modified from within functions.

```
global_var = 30 # global variable  
def print_global():  
    print(global_var)  
print_global()
```

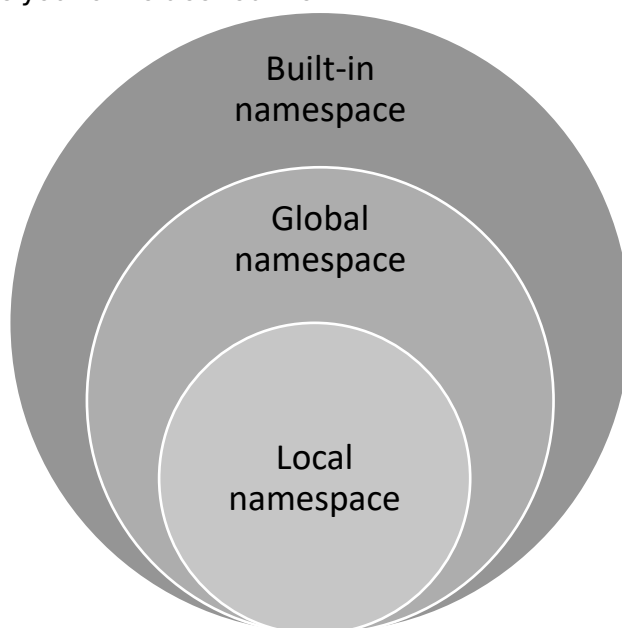
- **Built-in Scope:**

- python has a built-in scope that includes names like print, len, etc.
- These names are always accessible from any part of the code.

```
my_list = [1, 2, 3, 4, 5]  
# Using the built-in function len  
list_length = len(my_list)  
Print("Length of the list:", list_length)
```

- **Namespaces:**

A namespace in Python is a system that assigns a unique name to each object, such as variables or methods. Python maintains namespaces using dictionaries. Think of it like a directory-file system, where multiple files can have the same name in different directories, but specifying the absolute path directs you to the desired file.



- **Lifetime of Namespace:**

A lifetime of a namespace depends upon the scope of objects, if the scope of an object ends, the lifetime of that namespace comes to an end. Hence, it is not possible to access the inner namespace's objects from an outer namespace.

```
#num global is in the global namespace
num_global = 10
def outer _ function():
    #num2 is in the local namespace
    num_local = 20
    #num_nested is in the nested local namespace
    def inner _ function():
        num_nested = 30
    # Example usage:
    print("Global variable:", num_global)
outer_function()
```

1.3.2 Python Modules

- **Modules:**

A module in Python is a file containing Python code that defines variables, functions, and classes. These files typically have a .py extension. The primary purpose of modules is to organize code into reusable and logically structured units. Modules serve several key purposes:

1. **Code Organization:** Modules help organize code by grouping related functionality together. This enhances the maintainability and readability of *large codebases*.
2. **Code Reusability:** Functions, variables, and classes defined in a module can be reused in other Python scripts or modules. This promotes a modular and efficient coding style.
3. **Encapsulation:** Modules allow encapsulation of related code, hiding the internal details of the implementation. This supports the principle of information hiding and abstraction.
4. **Importing:** Modules are brought into a Python script using the import statement. This statement makes the names defined in the module accessible in the importing script, providing a way to use external code seamlessly.

e.g.:-

```
import math #This is called Built-in module.
```

```
Import my_module #This is called user-defined module.
```

➤ Unit 1.4: Data Structures in Python

1.4.1 Python Lists

- The list is a most versatile datatype available in Python which can be written as a list of comma-separated values (items) between square brackets.
- Lists are just like dynamic sized arrays, declared in other languages (vector) in C++ and ArrayList in Java). Lists need not be homogeneous always.
- Lists are mutable, and hence, they can be altered even after their creation.

e.g.(code):-

```
list1 = [1, 2, 3, 4, 5] #list of numbers
```

```
list2 = ["a", "b", "c", "d"] #list of strings
```

```
list3 = ['Physics', 'Chemistry', 123, 321] #list of strings and numbers
```

- **Accessing values from a list:**

To access values in lists, use the square brackets for slicing along with the index or indices to obtain value available at that index.

e.g.(code):-

```
list1 = ['Python', 'java', 123, 321]
```

```
list2 = [1, 2, 3, 4, 5]
```

```
print("list1[0]: ", list1[0]) #accessing elements at index 0
```

```
print("list2[1:5]: ", list2[1:5]) #accessing elements from indices 1 to 4
```

Output:

```
List1[0]: Python
```

```
List2[1:5]: [2, 3, 4, 5]
```

- **List Methods:**

Methods	Description
<code>list.append(obj)</code>	Appends object obj to list.
<code>list.count(obj)</code>	Returns count of how many times obj occurs in list.
<code>list.extend(seq)</code>	Appends the contents of seq to list.
<code>list.index(obj)</code>	Returns the lowest index in list that obj appears.
<code>list.insert(index, obj)</code>	Inserts object obj into list at offset index.
<code>list.pop(obj=list[-1])</code>	Removes and returns last object or obj from list.
<code>list.remove(obj)</code>	Removes object obj from list.
<code>list.reverse()</code>	Reverses objects list in place.
<code>list.sort([func])</code>	Sorts objects of list, use compare func if given.

- **Updating and Appending:**

To access values in lists, use the square brackets for slicing along with the index or indices to obtain value available at that index.

e.g.(code):-

```
list1 = ['Python', 'Java', 123, 321]
```

```
print("Value available at index 2 : ", list1[2])
```

```
list1[2] = 200
```

```
print("New value at index 2 : ", list1[2])
```

```
list1.append(400)
```

```
print(list1)
```

Output:

```
Value available at index 2 : 123
```

```
New value at index 2 : 200
```

```
['Python' , 'Java' , 200, 321]
```

- **Delete List Elements:**

To remove a list element, you can use either the del statement if you know exactly which element(s) you are deleting or the remove() method.

e.g.(code):-

```
list1 = ['Python', 'Java', 123, 321]
```

```
print("Printing the list : " list1)
```

```
del list1[2]      #deleting element using del keyword.
print("After deleting value at index 2 using del keyword.")
print(list1)
list1.remove('Java')      #deleting element using remove method.
print("After deleting element with value Java using remove method.")
print(list1)
```

Output:

```
Printing the list : ['Python', 'java', 123, 321]
After deleting value at index 2 using del keyword.
['Python', 'Java', 321]
After deleting element with value java using remove method.
['Python', 321]
```

- **Join and split Method:**

In Python, we can use the function split() to split a string and join() to join a string.
e.g.(code):-

```
# Split the string into list of strings
string = "LEARNING INITIATIVE FOR DATA ANALYSIS"
List1 = string.split(' ')
print(List1)
#Join the list of string into a string
List2 = ['He', 'will', 'play.']
string = ' '.join(List2)
print(string)
```

Output:

```
['LEARNING', 'INITIATIVE',
'FOR', 'DATA', 'ANALYSIS']
```

He will play.

- **Nested Lists:**

A list can contain any sort of object, even another list (sublist), known as nested list.
e.g.(code):-

```
list1 = [1, 2, ['3 (a)', '3 (b)'], 4, 5, 6]
print(list1)
#Access Nested list item by index
Print(list1[2][0], list1[2][1])
```

Output:

```
[1, 2, ['3 (a)', '3 (b)'], 4, 5, 6]
3 (a) 3(b)
```

- **Iterate through Nested List:**

A list can contain any sort of object, even another list (sublist), such kind of list is known as nested list.

e.g.(code):-

```
list1 = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for sublist in list1:
    for element in sublist:
        print(element)
```

Output:

```
1      2      3      4      5      6      7
8      9
```

- **Processing Nested Lists:**

We can use various list methods to process nested lists.

e.g.(code):-

```
#add item to a nested list.
list1 = ['a', ['bb', 'cc'], 'd']
list1[1].append('WW')
print(list1)
```

Output:

```
['a', ['bb', 'cc', 'WW'], 'd']
['a', ['WW', 'bb', 'cc', 'WW'], 'd']
```

```
#insert items at a specific position in a nested list.
```

```
list1[1].insert(0,'WW')
print(list1)
```

1.4.2 Python Tuples

Python tuples are immutable, ordered collections of elements, similar to lists, but defined using **parentheses**, and their elements cannot be modified after creation.

- A tuple is an ordered collection of values.
 - Tuples are ordered - They maintain a left-to-right positional ordering.
 - Accessed by index - Items in a tuple can be accessed using an index.
 - Tuples can contain any sort of object - It can be numbers, strings, lists and even other tuples.
 - Tuples are **immutable** - you can't add, delete, or change items after the tuple is defined.
- e.g.(code):-

```
tuple1 = (1, 2, 3)           #tuple of integers
tuple2 = ('Hello', 'Hi', 'Hey') #tuple of strings
tuple3 = (1, 'Hello', 1.23, True) #tuple of floating-point numbers, strings and boolean
print(tuple1)
print(tuple2)
print(tuple3)
```

Output:

```
(1, 2, 3)
('Hello', 'Hi', 'Hey')
(1, 'Hello', 1.23, True)
```

- **Mutable Objects and Immutable Objects:**

A **mutable** object is a changeable object that can be modified after it is created. e.g.-Lists

An **immutable** object is an object whose state can never be modified after it is created. e.g.-Tuples

- **Nested Tuples:**

A tuple can contain any sort of object, even another tuple (sub-tuple), known as nested tuple.

e.g.(code):-

```
T = ('red', ('green', 'blue'), 'yellow')
print(T) #Nested Tuple
#Access sub tuple items by index
print(T[1][0])
print(T[1][1])
```

Output:

```
('red', ('green', 'blue'), 'yellow')
Green
blue
```

- **Singleton Tuple:**

If you have only one value in a tuple, you can indicate this by including a trailing comma, just before the closing parentheses.

e.g.(code):-

```
#Singleton tuple
Tuple = (4,) #type TUPLE
#Not a tuple
Tuple1 = (4) #type INT
```

- **Tuple Slicing:**

To access a range of items in a tuple, you need to slice a tuple using a slicing operator. Tuple slicing is similar to list slicing.

e.g.(code):-

```
T = ('a', 'b', 'c', 'd', 'e', 'f')
print(T[2:5]) #prints from index 2 to 4
print(T[3:-1]) #prints from index 3 to -1 (i.e. 2nd last element).
```

Output:

```
('c', 'd', 'e')
('d', 'e')
```

- **Tuple Packing and unpacking:**

- When a tuple is created, the items in the **tuple are packed** together into the object.
- When a packed tuple is assigned to a new tuple, the individual items are **unpacked**. When unpacking, the number of variables on the left must match the number of items in tuple.

Tuple unpacking can be used as:

- Unpack tuples into variables
- Pass multiple arguments to a function
- Return multiple values from a function

e.g.(code):-

```
T = ('red', 'green', 'blue', 'cyan')
```

```
(a, b, c, d) = T
```

```
print(a, b, c, d)
```

Output:

red green blue cyan

- **Tuple Methods:**

Python has a set of built-in methods that you can call on tuple objects.

Methods	Description
<code>count()</code>	Returns the count of specific item in the tuple.
<code>index()</code>	Returns the index of the first instance of the specified item.
<code>all()</code>	Returns True if all tuple items are true.
<code>any()</code>	Returns True if any tuple item is true.
<code>enumerate()</code>	Takes a tuple and returns as enumerate object.
<code>len()</code>	Returns the number of items in the tuple.
<code>max()</code>	Returns the largest item of the tuple.
<code>min()</code>	Returns the smallest item of the tuple.
<code>sorted()</code>	Returns a sorted tuple.
<code>sum()</code>	Sums items of the tuple.
<code>tuple()</code>	Converts an iterable (list, string, set etc.) to a tuple

1.4.3 Python Sets

Python sets are unordered, mutable collections of unique elements, defined using **curly braces** or the **set() constructor**, allowing for operations like union, intersection, and difference. Let us now explore more about Sets.

- Python set is an unordered collection of unique items. They are commonly used for computing mathematical operations such as union, intersection, difference, and symmetric difference.
- Set items are unique Duplicate items are not allowed.
- Sets are unindexed — You cannot access set items by referring to an index.
- Sets are changeable (**mutable**) — They can be changed in place, can grow and shrink on demand.

e.g.(code):-

```
#sets don't allows duplicates. They automatically removed during creation of set.
```

```
S = {'red', 'green', 'blue', 'red'}
```

```
print(S)
```

```
#A set itself is mutable, but I can't contain mutable objects.
```

```
S1 = {[1,2], 'abc', 2,12}
```

```
print(S)
```

Output:

```
{ 'red' , 'green' , 'blue' , 'red' }
```

TypeError

- **Set Methods:**

Methods	Description
<code>add()</code>	Add a single item to a set.
<code>update()</code>	Add multiple items to a set.
<code>remove()</code>	Removes a single item from a set. Raise a key error if needed.
<code>discard()</code>	Removes a single item from a set. No any key error.
<code>pop()</code>	Removes random item from a set and returns it.
<code>clear()</code>	Removes all elements from the set.
<code>all()</code>	Returns True if all items in a set are true.
<code>any()</code>	Returns True if any item in a set is true.
<code>enumerate()</code>	Takes a set and returns an enumerate object.
<code>len()</code>	Returns the number of items in the set.
<code>max()</code>	Returns the largest item of the set.
<code>min()</code>	Returns the smallest item of the set.
<code>sorted()</code>	Returns a sorted set.
<code>sum()</code>	Sums items of the set.

e.g.(code):_

```
S = {'red', 'green', 'blue'}
S.add('yellow') #add()
print(S)
S.update(['hi', 'bye']) #update()
print(S)
S.discard('red') #discard()
print(S)
S.remove('red') #remove
A = S.pop() #pop()
print(A)
S.clear() #clear()
print(S)
Set = {True, False, 1, 2, 3}
print(all(Set)) #all()
print(any(Set)) #any()
Enum_set = enumerate(Set)
print(list(Enum_set)) #enumerate() & list()
print(len(Set)) #len()
print(max(Set)) #max()
print(min(Set)) #min()
sort_set = sorted(Set) #sorted()
print(sort_set)
sum = sum(Set) #sum()
print(sum)
```

Output:

```
{'green', 'red', 'yellow', 'blue'}
{'bye', 'blue', 'yellow', 'green', 'hi', 'red'}
{'bye', 'blue', 'yellow', 'green', 'hi'}
TypeError
bye
set()
False
True
[(0, False), (1, True), (2, 2), (3, 3)]
4
3
False
[False, True, 2, 3]
6
```

○ **Iterate Through Set:**

To iterate over the items of a set, use a simple for loop.

e.g.(code):-

```
S = {'red', 'green', 'blue'}
For item in S:
    print(item)
```

Output:

```
blue
green
red
```

- Set Operations:
 - Union of the sets A and B is the set of all items in either A or B.
 - Intersection of the sets A and B is the set of items common to both A and B.
 - Difference of the sets A and B is the set of items present in A which are not in B.
 - Symmetric difference of sets A and B is the set of all elements in either A or B, but not in both.

e.g.(code):-

```
def union(A, B):
    # Returns the union of the two sets.
    return A | B
def intersection(A, B):
    # Returns the intersection of the two sets.
    return A & B
def symmetric_difference(A, B):
    # Returns the symmetric difference of the two sets.
    return A ^ B
def difference(A, B):
    # Returns the difference between two sets.
    return A - B
```

1.4.4 Python Dictionaries

Python dictionaries are mutable, unordered collections of unique key-value pairs, providing efficient access and modification of data.

A dictionary is a mapping between a set of indexes (*known as keys*) and a set of values. Each key maps to a value. The association of a key and a value is called a *key:value* pair or sometimes an item.

e.g.(code):-

```
D = {'name': 'John',
     'age': 25,
     'job': 'manager',
     'city': 'delhi',
     'email': 'john@mail.com'}
```

- **The dict() Constructor:**

You can convert two-value sequences into a dictionary with Python's dict() constructor. The first item in each sequence is used as the key and the second as the value.

e.g.(code):-

create a dictionary with a list of two item tuples orVice-versa.

```
L = [('name', 'john'), ('age', 25), ('job', 'manager')]
```

```
D = dict(L)
```

```
print(D)
```

Output: {'name': 'john', 'age': 25, 'job': 'manager'}

- You can use **dict()** function along with the **zip()** function, to combine separate lists of keys and values obtained dynamically at runtime.

e.g.(code):-

create a dictionary with list of zipped key/values

```
Keys = ['name', 'age', 'job']
```

```
Values = ['John', 25, 'manager']
```

```
D = dict(zip(Keys, Values))
```

```
print(D)
```

Output:

```
{'name': 'john', 'age': 25, 'job': 'manager'}
```

- To create a dictionary with default values for each key, `fromkeys()` method can be used.
e.g.(code):-

```
# Initialize dictionary with default value '0' for each
Keys = ['name', 'age', 'job']
defaultvalue = 0
D = dict.fromkeys(defaultvalue)
print(D)
```

Output:

```
{'name': 0, 'age': 0, 'job': 0}
```

- **Python Dictionary Comprehension:**

Dictionary comprehension is a way to build a new dictionary by applying an expression to each item in an iterable.

e.g.(code):-

```
D = {y: y**2 for y in range(4)}
print(D)
```

Output:

```
{0: 0, 1: 1, 2: 4, 3: 9}
```

➤ Unit 2.1: Object Oriented Programming Fundamentals.

2.1.2 Programming Paradigm

A programming paradigm defines the methodology of designing and implementing programs using the key features and building blocks of a programming language.

- **Procedural programming:**

Procedural programming is a programming paradigm that uses procedures, subroutines, or functions to organize code into modular and reusable units. It utilizes a sequence of instructions to perform tasks. It uses a linear top-down approach during the design of the program.

Examples include FORTRAN, ALGOL, COBOL, BASIC, Pascal, and c.

- **Object—Oriented programming:**

Object-Oriented programming (OOP) is centered on objects, which encapsulate data through attributes and functionality through methods. These objects communicate via methods, and the program structure is built on classes, inheritance, and polymorphism.

Examples include Java, Python, PHP, JavaScript, Ruby, Objective-C, and Scala.

- **Object based programming:**

Object-Based Programming is centered on the concept of objects that encapsulate data and behavior. It is a sub-set of Object-oriented paradigms but lacks features like inheritance and polymorphism.

Examples include JavaScript, VB, Ado, and Fortran 90.

- **Object Oriented Programming- Characteristics:**

1. **Modularity:** Modularity refers to the act of partitioning the code into modules building them first followed by linking and finally combining them to form a complete project. Modularity ensures re-usability and thrives to minimize the duplication. A module in Python is nothing but a file containing Python definitions followed by methods & statements. The module name is generated out of the file name by removing the suffix “.py”.

2. **Classes and Objects:** Classes are blueprint for creating objects. Classes define functions called methods, which identify the behaviors and actions that an object created from the class can perform with its data. Object is an instance of a class. It is an entity that has state and behavior.

e.g.:-

```
class car():  
    #Making Objects  
    carl = car()  
    car2=car()  
    car3=car()
```

3. **Data Encapsulation:** Encapsulation is wrapping up of data and functions (that operate on the data) into a single unit (called class). Encapsulation also lead to data abstraction or hiding.
4. **Data Abstraction:** Data abstraction refers to, providing only essential information to the outside world and hiding their background details, to represent the needed information in program without presenting the details.
5. **Inheritance:** The capability of a class to derive properties and characteristics from another class.
- **Sub Class:** The class that inherits properties from another class is called Sub class or Derived Class.
 - **Super Class:** The class whose properties are inherited by sub class is called Base Class or Super class.
6. **Polymorphism:** Polymorphism means the ability to take various forms. In Python, Polymorphism allows us to define methods in the child class with the same name as defined in their parent class.
- Method overriding occurs when a child class redefines a method inherited from the parent class to better suit its own needs.

2.1.2 Classes and Objects

- **Class:**

A class is a user-defined blueprint or prototype from which objects are created. Classes provide a means of bundling data and functionality together.

e.g.(code):-

```
class car:  
    #Statement-1  
    Pass    #pass statement is used to indicate that class is empty.
```

- **`__init__()` Method and `self`:**

`__init__()` is a special method in Python that automatically runs when an object of a class is created, initializing the object. It serves as the constructor or initializer for the class.

When you define `__init__()` in a class definition, its first parameter should be `self`. `Self`-parameter refers to the individual object itself. It is used to fetch or set attributes of the particular instance.

e.g.(code):-


```
class Car:
    #initializer
    def __init__(self):
        pass
```

- **Attributes:**

Attributes are the characteristics or properties that describe an object or entity. They determine the appearance, state, or other qualities of that object.

There are two types of attributes-

- **Instance attribute:**

Instance attributes are specific to each instance of the class. Each object or instance created from the class can have different values for these attributes.

e.g.(code):-

#A class with two instance attributes

```
class Car:
    #initializer with instance attributes
    def __init__(self, color, style):
        self.color = color
        self.style = style
```

- **Class attribute:**

The class attribute is a shared variable among all objects of a class. Changes to this variable affect all instances.

e.g.(code):-

#A class with one class attribute

```
class Car:
    #class attribute
    Wheels = 4
    #initializer with instance attributes
    def __init__(self, color, style):
        self.color = color
        self.style = style
```

- **Methods:**